AI-ASSISTED COING
Assignment-18.1

Deepthi
B-32
2303A52390

Task 1 – Weather API Integration
Connect to a Weather API to fetch current weather details for a given city.
Instructions:
• Prompt AI to generate Python code using the requests library.
• Handle errors: invalid city name, API key errors, and network failures.
• Display temperature, humidity, and weather description in a readable format.
Expected Output:
• A working Python script that fetches and displays weather data with proper error handling.

Prompt:
Generate a python code for fetching weather data
From Open-meteo API and display it in a user-friendly format.
Use GeoCoding API to get the latitude and longitude of a city entered by the user.
If API Works Show Weather Data Else Show Error Message Like "Service Temporary Unavailable"
Handle The Errors Gracefully and Inform the User About It.

Code:
```
import requests

def get_coordinates(city):
    try:
        url = f"https://geocoding-api.open-
meteo.com/v1/search?name={city}"
        response = requests.get(url, timeout=5)
        response.raise_for_status()
        data = response.json()
        if data.get('results') and len(data['results']) > 0:
            return data['results'][0]['latitude'],
data['results'][0]['longitude']
        else:
            print("City not found.")
            return None
    except requests.exceptions.Timeout:
        print("Request timed out. Please try again.")
        return None
    except requests.exceptions.ConnectionError:
```

```
        print("Connection error. Please check your internet
connection.")
        return None
    except requests.exceptions.HTTPError:
        print("Service Temporary Unavailable")
        return None
    except ValueError:
        print("Invalid response format.")
        return None
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
        return None

def get_weather(latitude, longitude):
    try:
        url = f"https://api.open-
meteo.com/v1/forecast?latitude={latitude}&longitude={longitude}&current
_weather=true"
        response = requests.get(url, timeout=5)
        response.raise_for_status()
        data = response.json()
        return data['current_weather']
    except requests.exceptions.Timeout:
        print("Request timed out. Please try again.")
        return None
    except requests.exceptions.ConnectionError:
        print("Connection error. Please check your internet
connection.")
        return None
    except requests.exceptions.HTTPError:
        print("Service Temporary Unavailable")
        return None
    except ValueError:
        print("Invalid response format.")
        return None
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
        return None

def main():
    try:
        city = input("Enter the city name: ").strip()
        if not city:
```

```
            print("City name cannot be empty.")
            return

        coordinates = get_coordinates(city)
        if coordinates:
            weather = get_weather(*coordinates)
            if weather:
                print(f"\nCurrent weather in {city}:")
                print(f"Temperature: {weather['temperature']}°C")
                print(f"Wind Speed: {weather['windspeed']} km/h")
                print(f"Weather Code: {weather['weathercode']}")
        else:
            print("Failed to retrieve weather data.")
    except KeyboardInterrupt:
        print("\nProgram interrupted by user.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

if __name__ == "__main__":
    main()
```

Output:

```
Enter the city name: Warangal

Current weather in Warangal:
Temperature: 29.2°C
Wind Speed: 3.3 km/h
Weather Code: 3
```

Task 2 – Currency Exchange Rates
Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.
Instructions:
• Use AI to generate API request code.
• Validate user input for amount and currency codes.
• Implement error handling for invalid responses and API downtime.
• Show conversion results clearly in tabular format.
Expected Output:
• A script that converts INR to multiple currencies with graceful error handling.

Prompt:
Generate a python code for converting INR to other currencies
Use a free currency exchange API
Take amount as input from user
Convert INR to USD, EUR, and GBP
Display output in a clear table format
If API fails show message "Service Temporary Unavailable"
Handle invalid input and errors properly

Code:

```
import requests
def get_exchange_rates():
    try:
        url = "https://api.exchangerate-api.com/v4/latest/INR"
        response = requests.get(url, timeout=5)
        response.raise_for_status()
        data = response.json()
        return data.get('rates', {})
    except requests.exceptions.Timeout:
        print("Request timed out. Please try again.")
        return None
    except requests.exceptions.ConnectionError:
        print("Connection error. Please check your internet
connection.")
        return None
    except requests.exceptions.HTTPError:
        print("Service Temporary Unavailable")
        return None
    except ValueError:
        print("Invalid response format.")
        return None
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
```

```python
        return None
def convert_inr(amount, rates):
    try:
        usd = amount * rates.get('USD', 0)
        eur = amount * rates.get('EUR', 0)
        gbp = amount * rates.get('GBP', 0)
        return usd, eur, gbp
    except Exception as e:
        print(f"An error occurred during conversion: {e}")
        return None
def main():
    try:
        amount_str = input("Enter the amount in INR: ").strip()
        if not amount_str.replace('.', '', 1).isdigit():
            print("Invalid input. Please enter a numeric value.")
            return

        amount = float(amount_str)
        rates = get_exchange_rates()
        if rates:
            usd, eur, gbp = convert_inr(amount, rates)
            print(f"{'Currency':<10} {'Amount':>10}")
            print(f"{'USD':<10} {usd:>10.2f}")
            print(f"{'EUR':<10} {eur:>10.2f}")
            print(f"{'GBP':<10} {gbp:>10.2f}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    main()
```

Output:

```
Enter the amount in INR: 1000
Currency        Amount
USD              10.60
EUR               9.19
GBP               7.95
```

Task 3 – GitHub Repository Info Fetcher
Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).
Instructions:
• Prompt AI to write a Python function to send GET requests to GitHub API.
• Handle rate limit errors, 404 (repo not found), and invalid input.
• Display repository name, description, stars, forks, and open issues.
Expected Output:
• A Python program that retrieves and displays GitHub repository information with robust error handling.

Prompt:

Generate a python code for fetching repository details
Use a free API to get repository information
Take repository name as input (owner/repo)
Display name, description, stars, forks, and open issues
If repository not found show proper message
Handle API errors and invalid input

Code:

```
import requests
def get_repository_info(repo_name):
    try:
        url = f"https://api.github.com/repos/{repo_name}"
        response = requests.get(url, timeout=5)
        if response.status_code == 404:
            print("Repository not found.")
            return None
        response.raise_for_status()
        data = response.json()
        name = data.get('name', 'N/A')
        description = data.get('description', 'N/A')
        stars = data.get('stargazers_count', 'N/A')
        forks = data.get('forks_count', 'N/A')
        open_issues = data.get('open_issues_count', 'N/A')
        return name, description, stars, forks, open_issues
    except requests.exceptions.Timeout:
        print("Request timed out. Please try again.")
        return None
    except requests.exceptions.ConnectionError:
```

```
        print("Connection error. Please check your internet
connection.")
        return None
    except requests.exceptions.HTTPError as e:
        status_code = e.response.status_code if e.response is not None
else 'unknown'
        print(f"HTTP error while contacting GitHub API: {status_code}")
        return None
    except ValueError:
        print("Invalid response format.")
        return None
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
        return None
def main():
    try:
        repo_name = input("Enter the repository name (owner/repo):
").strip()
        if '/' not in repo_name:
            print("Invalid input. Please enter in the format
'owner/repo'.")
            return

        info = get_repository_info(repo_name)
        if info:
            name, description, stars, forks, open_issues = info
            print(f"Name: {name}")
            print(f"Description: {description}")
            print(f"Stars: {stars}")
            print(f"Forks: {forks}")
            print(f"Open Issues: {open_issues}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    main()
```

Output:

```
Enter the repository name (owner/repo): FreeCodeCamp/freecodecamp
Name: freeCodeCamp
Description: freeCodeCamp.org's open-source codebase and curriculum. Learn math, programming, and computer science for free.
Stars: 438816
Forks: 43768
Open Issues: 225
```

Task 4 – Real-Time Application: News Headlines Aggregator
Scenario: Build a News Aggregator using a free News API.
Requirements:
• Fetch top 5 headlines for a given category (sports, technology, health).
• Handle errors like invalid category, missing API key, and request failures.
• Display headlines in a user-friendly numbered list format.
• Include a retry mechanism if the first request fails.
Expected Output:
• A working Python script that retrieves and displays news headlines with proper error handling

Prompt:
Generate a python code for fetching news headlines
Use https://newsapi.org/ API
Take category as input (sports, technology, health)
Display top 5 headlines in numbered format
If API fails show message "Service Temporary Unavailable"
Handle invalid category and errors properly
Retry once if request fails

Code:

```
import requests
def get_exchange_rates():
    try:
        url = "https://api.exchangerate-api.com/v4/latest/INR"
        response = requests.get(url, timeout=5)
        response.raise_for_status()
        data = response.json()
        return data.get('rates', {})
    except requests.exceptions.Timeout:
        print("Request timed out. Please try again.")
        return None
    except requests.exceptions.ConnectionError:
        print("Connection error. Please check your internet
connection.")
        return None
    except requests.exceptions.HTTPError:
        print("Service Temporary Unavailable")
        return None
    except ValueError:
        print("Invalid response format.")
        return None
    except Exception as e:
```

```python
        print(f"An unexpected error occurred: {e}")
        return None
def convert_inr(amount, rates):
    try:
        usd = amount * rates.get('USD', 0)
        eur = amount * rates.get('EUR', 0)
        gbp = amount * rates.get('GBP', 0)
        return usd, eur, gbp
    except Exception as e:
        print(f"An error occurred during conversion: {e}")
        return None
def main():
    try:
        amount_str = input("Enter the amount in INR: ").strip()
        if not amount_str.replace('.', '', 1).isdigit():
            print("Invalid input. Please enter a numeric value.")
            return

        amount = float(amount_str)
        rates = get_exchange_rates()
        if rates:
            usd, eur, gbp = convert_inr(amount, rates)
            print(f"{'Currency':<10} {'Amount':>10}")
            print(f"{'USD':<10} {usd:>10.2f}")
            print(f"{'EUR':<10} {eur:>10.2f}")
            print(f"{'GBP':<10} {gbp:>10.2f}")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    main()
```

Output:

```
Enter news category (sports, technology, health): health
1. The surprising link between back pain and a sensitivity to loud noises - The Washington Post (Source: The Washington Post, Published At: 2026-03-24T14:22:24Z)
2. Exercise can lower Alzheimer's risk. Scientists may have discovered why. - The Washington Post (Source: The Washington Post, Published At: 2026-03-24T10:32:01Z)
3. Woman, 41, Was Tired of 'Hiding' Excess Skin After Losing 180 Lbs. on a GLP-1. Then She Got an 'Awake' Arm Lift (Exclusive) - Yahoo Life UK (Source: PEOPLE, Published At: 2026-03-24T08:30:00Z)
4. 2 Utah companies are quietly changing how AI actually works for real businesses - KSL.com (Source: KSL.com, Published At: 2026-03-24T03:02:25Z)
5. Psychology says people who describe themselves as joyful after 50 didn't suddenly become optimistic — they stopped treating happiness like a reward for perfect behavior and started treating it li
ke a practice - VegOut (Source: Vegoutmag.com, Published At: 2026-03-24T01:45:21Z)
```